

Part A: Timing Contains

Input Size (n)	Vector	List	Hash	Tree
1000	2.9195e-5 ms	2.8575e-5 ms	0.00013804 ms	0.00522394 ms
2000	3.026e-05 ms	2.0755e-05 ms	9.44e-05 ms	0.010031 ms
4000	2.1055e-05 ms	2.0995e-05 ms	0.000181135 ms	0.0254356 ms
8000	2.4615e-05 ms	2.081e-05 ms	0.00012416 ms	0.0774705 ms

Part B: Timing Query

Test 1: CompareItemName

Input Size (n)	Vector	List	Hash	Tree
1000	0.0612752 ms	0.0625814 ms	0.0463915 ms	0.03287 ms
2000	0.170935 ms	0.0922421 ms	0.110283 ms	0.0927156 ms
4000	0.19344 ms	0.189616 ms	0.207748 ms	0.138717 ms
8000	0.592586 ms	0.708092 ms	0.739506 ms	0.470728 ms

Test 2: CompareItemWeight

Input Size (n)	Vector	List	Hash	Tree
1000	0.0068285 ms	0.0083844 ms	0.0088885 ms	0.0013019 ms
2000	0.0126431 ms	0.0149484 ms	0.0149484 ms	0.0016573 ms
4000	0.0228954 ms	0.0310143 ms	0.0397664 ms	0.0031318 ms
8000	0.0613722 ms	0.0658826 ms	0.101715 ms	0.0073987 ms

Report: The results in the tables are what I expected. This is because vectors and lists should have similar times since they require checking each element sequentially, with a time complexity of $O(n)$. However, the “contains” operation’s times show that they are not growing linearly. This can be explained in the case that the program found the element before it reaches the end of the vector and list. On the other hand, the “query” operation’s times show that both are growing linearly as expected. Hash is expected to have a constant time look-up ($O(1)$), no matter the input size. The time supports that in Part A, as it remained around 0.0001 ms. In Part B, the times must have been growing slowly because it requires several lookups over a range of elements. The

difference in times can be explained by the fact that hash tables are only fast for a single lookup of an element. For the AVL tree, the times in Part A and B support logarithmic growth since it is growing slower than linearly. Based on these results, you would prefer a hash-based inventory if fast, constant-time insertion, deletion, and lookup operations of a single element are required. In the case where fast range query operations are necessary, a tree-based inventory should be used, as it performed the best against the other containers.